# ICT162

**Proctored Online Exam - July Semester 2025**

## Object Oriented Programming

**Thursday, 20 November 2025 04:00 pm - 06:00 pm**

---

**Time allowed: 2 hours**

---

**INSTRUCTIONS TO STUDENTS:**

1. This examination contains **11** questions.
2. Answer all parts of each question in the corresponding answer box.
3. All answers must be typed in the answer box and submitted in Examena.
4. This is a/an **Closed-book** examination.
5. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.
6. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.
7. **One** A4-sized (double-sided) hardcopy Cheat Sheet allowed.

**At the end of the examination.**
To end the examination, please click on the "Submit and end exam" button to submit your answers.

**THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.**

ICT162 Copyright © 2025 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

## Question 1

Demonstrate your understanding of class hierarchy, what will be the output of the following code snippet?

```python
class Parent:
    def __init__(self):
        self.__value = 5

    def display(self):
        return f"Parent value: {self.__value}"

class Child(Parent):
    def __init__(self):
        super().__init__()
        self.__value = 10

    def display(self):
        return f"{super().display()}  Child value: {self.__value}"

c = Child()
print(c.display())
```

A. Parent value: 5 Child value: 5
B. Parent value: 10 Child value: 10
C. Parent value: 5 Child value: 10
D. Parent value: 10 Child value: 5

(3 marks)

## Question 2

This question tests your understanding of attributes and methods. Which line/s will raise an error?

```python
1  class Rectangle:
2      def __init__(self, width, height):
3          self.__width = width
4          self.__height = height
5
6      @property
7      def width(self):
8          return self.__width
9
10     @property
11     def height(self):
12         return self.__height
13
14     @property
15     def area(self):
16         return self.width * self.height
17
18 rect = Rectangle(5, 10)
19 rect.width = 10
20 print(rect.area())
```

A. Lines 16
B. Line 19 and 20
C. Line 20
D. Line 18

(3 marks)

## Question 3

Demonstrate your understanding of object composition. Which line demonstrates composition?

```python
class Engine:
    def __init__(self, power):
        self.__power = power

    def start(self):
        return "Engine starts."

class Car:
    def __init__(self, model, engine):
        self.__model = model
        self.__engine = engine

    def start(self):
        return f"{self.__model} car is starting. {self.__engine.start()}"

engine = Engine("150hp")
car = Car("Sedan", engine)
print(car.start())
```

A. self.__model = model
B. self.__engine = engine
C. def start(self):
D. class Car:

(3 marks)

## Question 4

Apply OOP principle. Which part of the code enforces the requirement for subclasses to implement specific methods?

```python
from abc import ABC, abstractmethod

class AbstractVehicle(ABC):

    @abstractmethod
    def start(self):
        pass

class Bike(AbstractVehicle):
    def start(self):
        return "Bike is starting."

bike = Bike()
print(bike.start())
```

A. class AbstractVehicle(ABC):
B. @abstractmethod
C. class Bike(AbstractVehicle):
D. def start(self):

(3 marks)

## Question 5

Apply OOP principle. What will be printed when '**child_method**' is called?

```
class Parent:
    def parent_method(self):
        raise ValueError("An error occurred in Parent.")

class Child(Parent):
    def child_method(self):
        self.parent_method()
        raise ValueError("An error occurred in Child.")


child = Child()
try:
    child.child_method()
except ValueError as e:
    print(e)
```

A. An error occurred in Child. An error occurred in Parent.
B. An error occurred in Parent. An error occurred in Child.
C. An error occurred in Child.
D. An error occurred in Parent.

(3 marks)

## Question 6

Apply OOP principle. What will be the output when it is executed?

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass

class Dog(Animal):
    def sound(self):
        return "Bark"

class Cat(Animal):
    pass

animal = Cat()
print(animal.sound())
```

A. "Bark"
B. ""
C. TypeError: Can't instantiate abstract class Cat with abstract method 'sound'
D. AttributeError: 'Cat' object has no attribute 'sound'

(3 marks)

## Question 7

Apply OOP principle. Analyze the code below and determine which modification would correctly define the '**Bird**' class with the expected output: **"Flapping wings".**

```
from abc import ABC, abstractmethod

class Flying(ABC):
    @abstractmethod
    def fly(self):
        pass

class Bird(Flying):
    # Choose the correct solution
        __________

# Instantiate the class
bird = Bird()
print(bird.fly())
```

A. def fly(self): return "Flapping wings"
B. def flap(self): print("Flapping wings")
C. def fly(self): print("Flapping wings")
D. def flap(self): return "Flapping wings"

(3 marks)

## Question 8

Examine the following code intended to demonstrate method overloading that can accept 2 or 3 arguments. Identify the issue in the code that will prevent it from functioning as intended and propose a way to fix it.

```
class Operation:
    def multiply(self, x, y, z):
        return x * y * z

    def multiply(self, x, y):
        return x * y

op = Operation()
result = op.multiply(3, 4) # Expected output: 12
result = op.multiply(3, 4, 2) # Expected output: 24
```

Write your fixed code below.

```
class Operation:
 # your code here
```

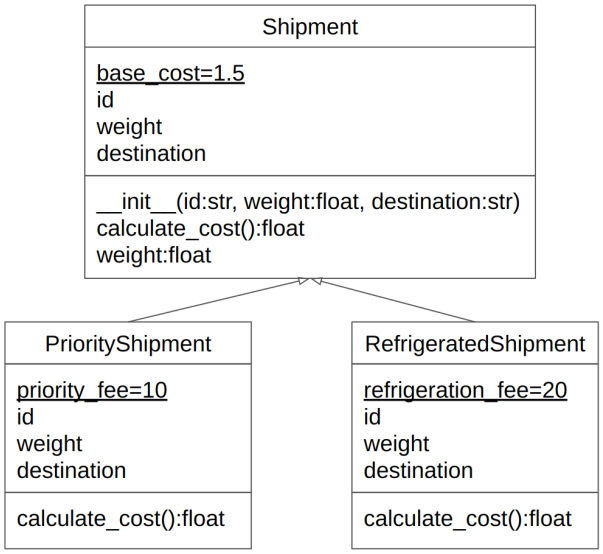
(6 marks)

## Question 9

**Context**: You are developing a system for a logistics company to manage different types of shipments. Each type of shipment calculates shipping cost differently based on the shipment's weight and includes a base cost associated with handling specific shipment types.

**Requirements:**
1. Create an abstract class 'Shipment' with an abstract method 'calculate_cost'.
2. Use class variables to define base costs and additional fees for each type of shipment.
3. Implement subclasses 'RefrigeratedShipment' and 'PriorityShipment' for different shipment types.

Class diagram:




**Important:**
Please refer to the Sample Code and Usage for the classess below. Your implementation should ensure the Sample Code can be executed without error and return the expected outputs.

**Sample Code and Usage**:

```
shipments = [
    PriorityShipment("101", 10, "New York"),
    RefrigeratedShipment("102", 5, "San Francisco"),
]

for shipment in shipments:
    print(shipment)
```

**Expected Output**:

```
Shipment ID 101 to New York: Cost = $25.00
Shipment ID 102 to San Francisco: Cost = $27.50
```

## Question 9a

Construct the class with the following specifications.
Define an abstract class named 'Shipment' with:
- An '__init__' method that accepts 'id', 'weight', and 'destination'.
- An abstract method 'calculate_cost'.
- A class variable 'base_cost' set to 1.5.
- Getter for 'weight' using decorator.

(15 marks)

## Question 9b

Construct the subclasses 'PriorityShipment', and 'RefrigeratedShipment'. Each class should:
- Implement the 'calculate_cost' method to include the additional fees.
- 'PriorityShipment': An additional priority_fee.
- 'RefrigeratedShipment': An additional refrigeration_fee.
- Expected outputs are provided in the question description above. You have to ensure that the Sample Code can run without any errors using your implementation.

(14 marks)

## Question 9c

Implement a '__str__' in the Shipment class. That fulfils the expected output shown in the question description.
Use 'id' and 'destination' to identify each shipment in the report. Please refer to the usage and expected output above.

(6 marks)

## Question 10

Building upon the previous system that manages different types of shipments ('PriorityShipment', 'RefrigeratedShipment') with various cost calculations, we will now extend this system to include a tracking system for shipments.
This tracking system will manage the status and history of each shipment.

### Requirements

You need to implement a new feature to track the status of shipments, using composition.
The 'Tracking' component has been implemented for you.

```python
class Tracking:
    def __init__(self):
        self.__history = []

    def update_status(self, status):
        # format datetime to display year, month and day
        formatted_datetime = datetime.now().strftime("%Y-%m-%d")
        self.__history.append((formatted_datetime, status))

    def get_history(self):
        if not self.__history:
            raise Exception("No tracking history available.")
        return self.__history
```

You will integrate 'Tracking' component into your existing 'Shipment' classes to enable the tracking of shipment statuses and history.

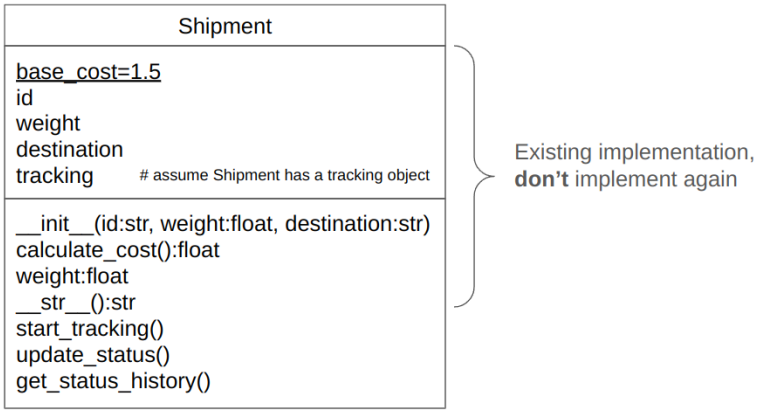
**Sample code to demonstrate usage and show expected outputs**:

```
# Tracking object behaviors
tracking = Tracking()
tracking.update_status("Initialized")
tracking.update_status("Awaiting Pickup")
print(f"Tracking class usage: {tracking.get_history()}")
```

**Expected Output** (assuming execution on a hypothetical date, e.g., 2024, 12, 29):

```
Tracking class usage: [('2024-12-29', 'Initialized'), ('2024-12-29', 'Awaiting Pickup')]
```

Modify the 'Shipment' class to incorporate 'Tracking' using composition. Implement additional 'start_tracking', 'update_status' and 'get_status_history' methods in the 'Shipment' class, implement exception handling where necessary.




Refer to the sample code with the expected output below, the implementation should allow the sample code to run successfully.

**Sample code to demonstrate usage and show expected outputs**:

```
# Create shipments
refrigerated_shipment = RefrigeratedShipment(103, 8, "Chicago")
# Begin tracking for shipments
refrigerated_shipment.start_tracking()
# Update statuses
refrigerated_shipment.update_status("Awaiting Pickup")
# Print status histories
print(f"Status History: {refrigerated_shipment.get_status_history()}")
```

**Expected Output** (assuming execution on a hypothetical date, e.g., 2024, 12, 29):

```
Status History: [('2024-12-29', 'Initialized'), ('2024-12-29', 'Awaiting Pickup')]
```

(22 marks)

## Question 11

You are tasked with developing a simple graphical user interface (GUI) for the logistics system to allow users to interact with shipment details and update their statuses.

The GUI will provide fields for viewing shipment information and buttons for updating status.

Requirement:
Use the Tkinter library to build a basic GUI application that allows users to select a shipment, view its details, and update its status.

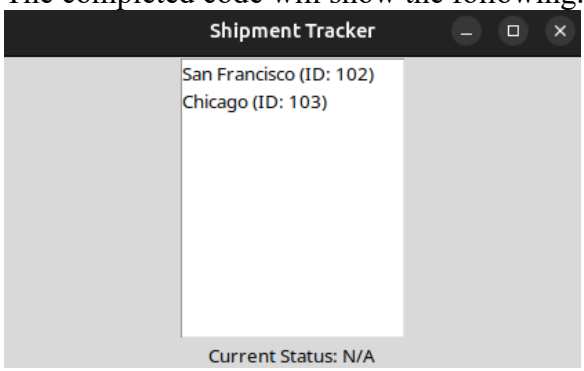
## Question 11a

The sample code shows a Tkinter window with a 'Listbox' to select a shipment and 'Label' widgets to display its current status and details.

Line 16 and 17 is missing, complete the code to create the GUI window.

```
1  import tkinter as tk
2
3  class ShipmentTrackerGUI():
4      # Create main window
5      def __init__(self, shipment_list):
6          win = tk.Tk()
7          win.title("Shipment Tracker")
8          self.__shipment_list = shipment_list
9          # Shipment Listbox
10         self.__listbox = tk.Listbox(win)
11         for shipment in self.__shipment_list:
12             self.__listbox.insert(tk.END, f"{shipment.destination} (ID: {shipment.id})")
13         # Label to display current status
14         self.__status_label = tk.Label(win, text="Current Status: N/A")
15
16         # missing line
17         # missing line
18
19         win.mainloop()
20
21 def main():
22     shipment_list = [
23         PriorityShipment(102, 5, "San Francisco"),
24         RefrigeratedShipment(103, 8, "Chicago")
25     ]
26
27     # Start tracking for shipments
28     for shipment in shipment_list:
29         shipment.start_tracking()
30
31     shipment_tracker = ShipmentTrackerGUI(shipment_list)
32
33 main()
```

The completed code will show the following.

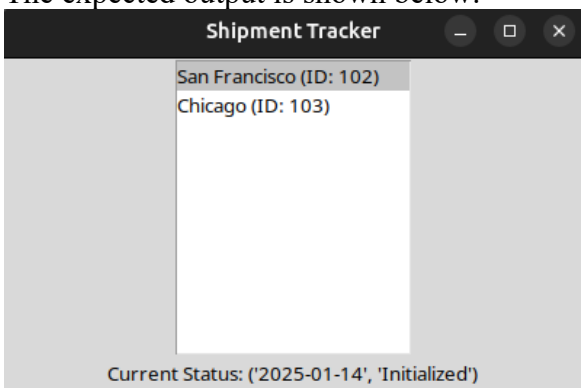



(8 marks)

**Question 11b**

Ensure the GUI can dynamically update the current status.
The goal is to display the current status, i.e. 'shipment.get_status_history()', when our cursor select the listbox.

The expected output is shown below.




The code is given below, however **ONE (1)** line is missing.

```
        self.__listbox.bind('<<ListboxSelect>>', self.on_select)

        win.mainloop()

    def on_select(self, event):
        # this will get the index of the selected item
        selected_index = self.__listbox.curselection()
        if selected_index:
            # get the shipment object from the list
            shipment = self.__shipment_list[selected_index[0]]
            # missing line
```

Please fill in the missing line in the above code.

(8 marks)

**----- END OF PAPER -----**